

TRABAJO PRÁCTICO INTEGRADOR (TPI)

Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación a Distancia

Materia: Programación 1

Título del proyecto: Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Alumno:

- Centurión Facundo Gabriel
- **Fecha de entrega:** 29/06/26

Índice

1. Introducción	3
2. Marco Teórico	3
○ 2.1 Listas	3
○ 2.2 Diccionarios	3
○ 2.3 Funciones	4
○ 2.4 Estructuras condicionales	4
○ 2.5 Estructuras repetitivas	4
○ 2.6 Ordenamientos	5
○ 2.7 Estadísticas básicas	5
○ 2.8 Archivos CSV	6
3. Decisiones Técnicas y Arquitectura	6
○ 3.1 Diagrama de flujo	6
○ 3.2 Estructura de datos elegida	7
○ 3.3 Modularización	7
○ 3.4 Validaciones implementadas	7
○ 3.5 Capturas de pantalla	8
4. Dificultades y Conclusiones	10
○ 4.1 Dificultades encontradas	10
○ 4.2 Conclusiones	10
5. Bibliografía / Webgrafía	11
6. Links del proyecto	12

1. Introducción

El presente trabajo práctico integrador corresponde a la materia Programación 1 de la Tecnicatura Universitaria en Programación de la Universidad Tecnológica Nacional. El objetivo fue desarrollar una aplicación en Python 3 que permita gestionar información sobre países, aplicando los conceptos aprendidos durante la cursada: listas, diccionarios, funciones, estructuras condicionales y repetitivas, ordenamientos, estadísticas y manejo de archivos CSV.

El sistema desarrollado permite al usuario agregar, actualizar, buscar, filtrar y ordenar países, además de visualizar estadísticas generales sobre el dataset cargado.

2. Marco Teórico

2.1 Listas

Una lista en Python es una estructura de datos ordenada y mutable que permite almacenar múltiples elementos en una sola variable. Los elementos se acceden mediante índices numéricos que comienzan en cero. En este proyecto, la lista es la estructura principal que contiene todos los países cargados desde el CSV.

Ejemplo:

```
python
países = ["Argentina", "Brasil", "México"]
print(países[0]) # Argentina
```

2.2 Diccionarios

Un diccionario es una estructura de datos que almacena pares clave-valor. A diferencia de las listas, los elementos se acceden mediante claves en lugar de índices. En este proyecto, cada país se representa como un diccionario con las claves nombre, población, superficie y continente.

Ejemplo:

```
python
pais = {
    "nombre": "Argentina",
    "poblacion": 45376763,
```

```
"superficie": 2780400,  
"continente": "América"  
}  
print(pais["nombre"]) # Argentina
```

2.3 Funciones

Una función es un bloque de código reutilizable que realiza una tarea específica. En Python se definen con la palabra clave `def`. En este proyecto se aplicó el principio de una función = una responsabilidad, dividiendo el programa en funciones independientes para cada operación.

Ejemplo:

```
python  
def saludar(nombre):  
    print(f"Hola, {nombre}")  
saludar("Ana") # Hola, Ana
```

2.4 Estructuras condicionales

Las estructuras condicionales permiten ejecutar distintos bloques de código según si una condición es verdadera o falsa. En Python se usan `if`, `elif` y `else`. En este proyecto se utilizan para validar entradas del usuario y para controlar el flujo del menú principal.

Ejemplo:

```
python  
edad = 20  
if edad >= 18:  
    print("Mayor de edad")  
else:  
    print("Menor de edad")
```

2.5 Estructuras repetitivas

Las estructuras repetitivas permiten ejecutar un bloque de código múltiples veces. Python ofrece `for` para iterar sobre secuencias y `while` para repetir mientras una

condición sea verdadera. En este proyecto se usa `while True` para mantener el menú activo y para repetir la solicitud de datos hasta que el usuario ingrese valores válidos.

Ejemplo:

```
python
while True:
    valor = input("Ingrese un número positivo: ")
    if valor.isdigit() and int(valor) > 0:
        break
    print("Valor inválido, intente de nuevo.")
```

2.6 Ordenamientos

El ordenamiento consiste en reorganizar los elementos de una colección según un criterio determinado. Python ofrece la función `sorted()` que devuelve una nueva lista ordenada sin modificar la original. Se puede ordenar de forma ascendente o descendente, y se puede especificar el criterio con el parámetro `key`.

Ejemplo:

```
python
países = [{"nombre": "Brasil", "poblacion": 213000000}, {"nombre": "Argentina",
"poblacion": 45000000}]
ordenados = sorted(países, key=lambda p: p["poblacion"], reverse=True)
```

2.7 Estadísticas básicas

Las estadísticas básicas permiten obtener indicadores numéricos sobre un conjunto de datos. En Python se pueden calcular fácilmente usando funciones como `max()`, `min()` y `sum()` combinadas con `len()` para obtener promedios.

Ejemplo:

```
python
poblaciones = [45000000, 213000000, 126000000]
promedio = sum(poblaciones) / len(poblaciones)
mayor = max(poblaciones)
```

2.8 Archivos CSV

CSV (Comma-Separated Values) es un formato de archivo de texto plano donde los datos se organizan en filas y columnas separadas por comas. Python incluye el módulo csv en su librería estándar, que facilita la lectura y escritura de este tipo de archivos. En este proyecto se usa csv.DictReader para leer cada fila como un diccionario y csv.DictWriter para escribir los datos de vuelta al archivo.

Ejemplo:

```
python
import csv

with open("países.csv", newline="", encoding="utf-8") as archivo:
    lector = csv.DictReader(archivo)
    for fila in lector:
        print(fila["nombre"])
```

3. Decisiones Técnicas y Arquitectura

3.1 Diagrama de flujo

El flujo general del programa es el siguiente:

INICIO

|

▼

Cargar países.csv

|

▼

Mostrar menú principal

|

├── 1. Agregar país ────> Validar datos ────> Agregar a lista ────> Guardar CSV

├── 2. Actualizar país ────> Buscar país ────> Validar datos ────> Guardar CSV

├── 3. Buscar por nombre ────> Mostrar resultados

├── 4. Filtrar ────> Submenú filtros ────> Mostrar resultados

├── 5. Ordenar ────> Submenú orden ────> Mostrar resultados

├── 6. Estadísticas ────> Calcular y mostrar

└── 0. Salir ────> FIN

3.2 Estructura de datos elegida

Se optó por representar el dataset como una lista de diccionarios, donde cada diccionario representa un país con sus cuatro atributos. Esta estructura permite acceder a los datos por nombre de campo (más legible que por índice), iterar fácilmente con `for`, y aplicar funciones como `sorted()`, `max()` y `min()` con el parámetro `key`.

3.3 Modularización

El programa se dividió en funciones con responsabilidad única:

Función	Responsabilidad
<code>cargar_paises()</code>	Leer el CSV y devolver la lista de países
<code>guardar_paises()</code>	Escribir la lista de países en el CSV
<code>agregar_pais()</code>	Solicitar datos y agregar un nuevo país
<code>actualizar_pais()</code>	Buscar un país y modificar sus datos
<code>buscar_pais()</code>	Buscar por nombre con coincidencia parcial
<code>filtrar_por_continente()</code>	Filtrar países por continente
<code>filtrar_por_poblacion()</code>	Filtrar por rango de población
<code>filtrar_por_superficie()</code>	Filtrar por rango de superficie
<code>ordenar_paises()</code>	Ordenar por nombre, población o superficie
<code>mostrar_estadisticas()</code>	Calcular y mostrar estadísticas del dataset
<code>mostrar_tabla()</code>	Mostrar una lista de países en formato tabla

3.4 Validaciones implementadas

- Ningún campo puede estar vacío al agregar un país.
- La población y superficie deben ser números enteros positivos.
- El continente debe ser uno de los valores válidos definidos.
- No se permite agregar un país con un nombre ya existente.

- Al cargar el CSV, las filas con formato inválido se omiten con un mensaje de advertencia.
- Las búsquedas sin resultados muestran un mensaje claro al usuario.
- El archivo CSV se guarda únicamente si la operación de agregar o actualizar fue exitosa.

3.5 Capturas de pantalla

```

Símbolo del sistema - python main.py
Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Centurion>cd C:\Users\Centurion\OneDrive\Escritorio\tpi-programacion1
C:\Users\Centurion\OneDrive\Escritorio\tpi-programacion1>python main.py

Cargando datos...
Se cargaron 21 países correctamente.

=====
GESTIÓN DE DATOS DE PAÍSES
=====
1. Agregar un país
2. Actualizar datos de un país
3. Buscar país por nombre
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
0. Salir
=====

Ingrese una opción:

```

Captura del programa corriendo en CMD

```

Símbolo del sistema - python main.py
Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Centurion>cd C:\Users\Centurion\OneDrive\Escritorio\tpi-programacion1
C:\Users\Centurion\OneDrive\Escritorio\tpi-programacion1>python main.py

Cargando datos...
Se cargaron 21 países correctamente.

=====
GESTIÓN DE DATOS DE PAÍSES
=====
1. Agregar un país
2. Actualizar datos de un país
3. Buscar país por nombre
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
0. Salir
=====

Ingrese una opción: 1

--- AGREGAR PAÍS ---
Nombre del país: Uruguay
Población: 3400000
Superficie en km²: 176215
Continentes disponibles: América, Europa, Asia, África, Oceanía
Continente: america

País 'Uruguay' agregado correctamente.
Datos guardados correctamente.

```

Captura ejecutando comando "agregar un país"


```
Símbolo del sistema - python main.py
0. Volver
Ingrese una opción: 1
--- FILTRAR POR CONTINENTE ---
Continentes disponibles: América, Europa, Asia, África, Oceanía
Ingrese el continente: america
Países en america (7):
Nombre                Población    Superficie (km²) Continente
-----
Argentina             45,376,763   2,780,400 América
Brasil                213,993,437   8,515,767 América
México               126,705,138   1,964,375 América
Colombia              51,874,024   1,141,748 América
Canadá                38,246,108   9,984,670 América
Estados Unidos       331,002,651   9,833,517 América
Uruguay              3,500,000    176,215 América

=====
GESTIÓN DE DATOS DE PAÍSES
=====
1. Agregar un país
2. Actualizar datos de un país
3. Buscar país por nombre
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
0. Salir
=====
Ingrese una opción:
```

Captura ejecutando comando “Filtrar países por continente AMERICA”

```
Símbolo del sistema - python main.py
Ingrese una opción: 5
--- ORDENAR POR ---
1. Nombre
2. Población
3. Superficie
0. Volver
Ingrese una opción: 2
--- DIRECCIÓN ---
1. Ascendente
2. Descendente
Ingrese una opción: 1
Países ordenados por Población (ascendente):
Nombre                Población    Superficie (km²) Continente
-----
Uruguay              3,500,000    176,215 América
Australia            25,499,884   7,692,024 Oceanía
Arabia Saudita       34,813,871   2,149,690 Asia
Canadá               38,246,108   9,984,670 América
Argentina            45,376,763   2,780,400 América
España              47,351,567   505,990 Europa
Colombia             51,874,024   1,141,748 América
Sudáfrica            59,308,690   1,219,090 África
Italia               60,367,477   301,340 Europa
Francia              67,391,582   551,695 Europa
Reino Unido          67,886,011   243,610 Europa
Alemania             83,149,300   357,022 Europa
Egipto              102,334,404   1,001,449 África
Japón               125,800,000   377,975 Asia
México              126,705,138   1,964,375 América
Rusia               144,104,080   17,098,242 Europa
Nigeria            206,139,589   923,768 África
Brasil             213,993,437   8,515,767 América
Indonesia           273,523,615   1,904,569 Asia
Estados Unidos     331,002,651   9,833,517 América
India              1,380,004,385   3,287,263 Asia
China              1,439,323,776   9,596,960 Asia

=====
```

Captura ejecutando comando “Ordenar por: POBLACION/ASCENDENTE”

4. Dificultades y Conclusiones

4.1 Dificultades encontradas

- Al principio no sabía cómo leer el CSV como diccionarios; investigando encontré **csv.DictReader**, que simplificó mucho el código.
- Tuve que aprender a usar `sorted()` con el parámetro `key` y una función lambda para ordenar por un campo específico del diccionario.
- El manejo de caracteres especiales como tildes y la ñ requirió especificar `encoding="utf-8"` al abrir los archivos.
- Al principio el menú volvía al inicio sin mostrar mensajes de error cuando el usuario ingresaba una opción inválida. Tuve que revisar el flujo del `while True` y agregar validaciones para cada caso posible.
- Me costó entender cómo funcionaba `csv.DictReader` al principio porque estaba acostumbrado a leer archivos línea por línea. Una vez que entendí que devuelve cada fila como un diccionario, el código se simplificó bastante.

4.2 Conclusiones

- Este proyecto me permitió integrar todos los conceptos vistos en la materia en una aplicación real y funcional.
- Aprendí la importancia de modularizar el código: dividir el programa en funciones pequeñas hace que sea más fácil de entender, corregir y ampliar.
- Modularizar el código en funciones con una sola responsabilidad no solo hace el programa más ordenado, sino que también facilita detectar y corregir errores, ya que sabés exactamente en qué función buscar cuando algo falla.
- Este proyecto me hizo entender la diferencia entre tener conocimientos teóricos y saber aplicarlos. Conocía las listas y los diccionarios por separado, pero combinarlos para representar datos del mundo real fue un desafío distinto que me obligó a pensar de otra manera.
- Desarrollar una aplicación completa desde cero, con menú, validaciones, lectura de archivos y estadísticas, me dio una visión más clara de cómo se estructura un programa real y me dejó con ganas de seguir aprendiendo para poder hacer proyectos más complejos.

5. Bibliografía / Webgrafía

- Python Software Foundation. *Documentación oficial de Python 3*. <https://docs.python.org/3/>
- Python Software Foundation. *Módulo csv*. <https://docs.python.org/3/library/csv.html>
- Python Software Foundation. *Funciones integradas: sorted, max, min, sum*. <https://docs.python.org/3/library/functions.html>
- Real Python. *Reading and Writing CSV Files in Python*. <https://realpython.com/python-csv/>
- Real Python. *Python Dictionaries*. <https://realpython.com/python-dicts/>

6. Links del proyecto

- **Repositorio GitHub:** <https://github.com/Facto91/tpi-programacion1>
- **Video demostrativo:** <https://youtu.be/DolC-qEHaCA>
- **Documentación PDF:** https://github.com/Facto91/tpi-programacion1/blob/main/Trabajo_Practico_Integrador_Programacion1_Centuri_on_Facundo.pdf